Объект хандалтат программчлалын удамшил, түүний зөв хэрэглээ

Лабораторийн ажил 7

МУИС, МТЭС, МКУТ-ийн суралцагч

Э. Баттамир, 24B1NUM0616@stud.num.edu.mn

А. Баяржавхлан, 24B1NUM0876@stud.num.edu.mn

Б. Маргад-Эрдэнэ, 24B1NUM1418@stud.num.edu.mn

7.1 Оршил

Объект хандалтат програмчлалын суурь тулгуур ойлголтуудын нэг болох удамшил нь бэлэн байгаа классын шинж чанар, функцуудыг өөр нэг класс өөртөө өвлөн авч, баяжуулан ашиглах боломжийг олгодог. Энэ нь кодын давхцлыг арилгах, програмын засвар үйлчилгээ хийх хугацааг хэмнэх бодит ач холбогдолтой.

Энэхүү лабораторийн ажлын хүрээнд C++ хэлний удамшлын механизмыг ашиглан Shape, Shape2D, Circle, Square, Triangle классуудыг шаталсан бүтцийг үүсгэж, полиморфизмын зарчмыг практик дээр хэрэгжүүлэхийг зорилоо. Мөн кодыг олон файлд хуваах (.h, .cpp), header файл болон Makefile ашиглан төслийг нэгтгэн эмхэтгэх, багийн гишүүд Github платформ дээр хамтран ажиллах арга барилд суралцахад энэ ажлын гол утга санаа оршино.

7.2 Зорилго

1. C++ хэлний объект хандалтат програмчлалын үндсэн ойлголтууд болох удамшил, хийсвэрлэл, нарийвчлал, полиморфизмын ойлголтыг судлах.
2. Shape суурь класст тулгуурлан Circle, Square, Triangle зэрэг дэд классуудыг үүсгэж, удамшлын зарчмыг практикт хэрэгжүүлэх.
3. Классуудын байгуулагч функцүүдийг тодорхойлж, удамшлын үед эх классын параметертэй байгуулагчийг ашиглан дуудаж ажиллуулах аргачлалыг эзэмших.
4. Олон объект үүсгэн тэдгээрийн талбайг тооцоолж, талбайгаар нь эрэмбэлэх замаар полиморфизм болон алгоритмын хэрэглээг ойлгох.
5. Эх классын байгуулагчийг дуудах аргын давуу болон сул талыг харьцуулан дүгнэх.
6. Кодын зохион байгуулалт, дахин ашиглалт болон гүйцэтгэлийн хувьд объект хандалтат программчлалын ач холбогдлыг ойлгох.

7.3 Онолын судалгаа

1. **Удамшил ба байгуулагч, устгагч функцийн хамаарал?**

   Удамшил ашигласан үед объект үүсэх болон устах дараалал тодорхой дүрэмтэй байдаг. Удамшсан классын объект үүсэхэд хамгийн түрүүнд эх классын байгуулагч функц дуудагддаг[1]. Үүний дараа хүүхэд классын байгуулагч ажиллана. Учир нь хүүхэд класс зөв ажиллахын тулд эх классын хэсэг нь эхлээд бүрэн үүссэн байх шаардлагатай. Харин объект устах үед эсрэг дараалалтай ажиллана. Өөрөөр хэлбэл, эхлээд хүүхэд классын устгагч функц дуудагдаж, дараа нь эх классын устгагч ажиллана[1]. Хэрэв эх класс параметртэй байгуулагчтай бол хүүхэд класс тухайн байгуулагчийг заавал дуудаж өгөх хэрэгтэй байдаг. Мөн заагч ашиглаж, удамшсан классуудтай ажиллах үед эх классын устгагчийг virtual гэж зарлах нь чухал[1]. Ингэснээр объект устах үед зөв дараалаар бүх классын устгагч функцүүд дуудагдаж, санах ойн цоорхой үүсэхээс сэргийлнэ[1].

2. **Ерөнхийлөл гэж юу вэ? c++ хэл дээр хэрхэн хэрэгжүүлдэг вэ?**

   Ерөнхийлөл гэдэг нь програмчлалд тухайн объектын гол, чухал шинж чанарыг нь үлдээн, хэрэгжүүлэлтийн нарийн деталиудыг нуух ойлголт юм. Өөрөөр хэлбэл, хэрэглэгч тухайн зүйлийг “юу хийж байгааг” мэдэх боловч “яаж хийж байгааг” мэдэх шаардлагагүй байдаг[1].
   C++ хэл дээр ерөнхийлөлийг ихэвчлэн хийсвэр класс болон виртуал функц ашиглан хэрэгжүүлдэг[1]. Abstract класс гэдэг нь дор хаяж нэг pure virtual функц (=0 гэж тодорхойлогдсон функц) агуулсан класс бөгөөд ийм классын объект шууд үүсгэж болдоггүй. Харин энэ классыг удамшуулсан хүүхэд классууд уг функцыг заавал өөрийнхөөрөө хэрэгжүүлдэг[1].

   Жишээ нь:
   ```
   class Shape2D : public Shape {
      public:
            Shape2D() : Shape() {};
            Shape2D(string name) : Shape(name) {}
            virtual double area() = 0;
            virtual double perimeter() = 0;
   };
   ```

3. **Нарийвчлал гэж юу вэ?**

   Нарийвчлал гэдэг нь объект хандалтат програмчлалд өгөгдөл болон түүнийг боловсруулах функцүүдийг нэг дор багцалж, гаднаас шууд хандахыг хязгаарлах ойлголт юм[3]. Өөрөөр хэлбэл, объектын дотоод мэдээллийг шууд өөрчлөхөөс хамгаалж, зөвхөн зөвшөөрөгдсөн функцээр дамжуулан хандах боломжийг олгодог[3].

Нарийвчлалыг private, protected, public ашиглан хэрэгжүүлдэг. Ихэвчлэн өгөгдлийг private болгож, харин түүнийг ашиглах getter, setter функцүүдийг public болгож өгдөг[3].

4. **Функц дахин программчлах гэж юу вэ? Жишээгээр тайлбарлах.**

Функц дахин программчлах гэдэг нь объект хандалтат програмчлалд удамшсан класс нь эх классын функцийг ижил нэр, ижил параметртэйгээр өөрийн хэрэгцээнд тохируулан дахин тодорхойлж хэрэгжүүлэхийг хэлнэ. Энэ нь полиморпизмын нэг хэлбэр юм[2].

Жишээ нь:

```
double area() override;
double perimeter() override;
```

7.4 Хэрэгжүүлэлт

1. Эх классуудад байгуулагч функц тодорхойлж удамшлын үед эх классын параметертэй байгуулагчыг дуудаж ажиллуулах.

```
Shape2D() : Shape() {};
Shape2D(string name) : Shape(name) {}

Circle() : Shape2D("Circle") {
radius = 0;
crd_1 = 0;
crd_2 = 0;
}
Circle(double r) : Shape2D("Circle") {
radius = r;
}

Square() : Shape2D("Square") {
side = 0;
}
Square(double s) : Shape2D("Square") {
side = s;
}

Triangle() : Shape2D("Triangle") {
side = 0;
}
Triangle(double a) : Shape2D("Triangle") {
side = a;
}

Shape2D *shapes[n] = {new Square(3), new Triangle(3), new Circle(3)};
```

```
double* k = new double[n];

for (int i = 0; i < n; i++) k[i] = shapes[i]->area();
```

Эх классын параметертэй байгуулагчийг удамшсан класст ашиглан дуудах нь объектын өгөгдлийг зөв, үр ашигтайгаар эхлүүлэх боломж олгодог. Энэ нь код дахин ашиглалт болон гүйцэтгэлийг сайжруулах давуу талтай боловч классуудын хоорондын хамаарлыг нэмэгдүүлж, бүтэц төвөгтэй болгох сул талтай. Иймээс зөв зохион байгуулалттай ашиглах нь чухал.

2. Лаб 6-д хийсэн circle, square, triangle классуудаас олон объект үүсгэж дүрс хооронд талбайгаар нь эрэмбэлэх.

```
int max(double *areas, int n) {
    int maxidx = 0;
    for (int i = 1; i < n; i++ ) {
        if (*(areas + i) > *(areas + maxidx)) maxidx = i;
    }

    return maxidx;
}

int x = max(k, n);

cout << "Hamgiin tom ni ";

switch (x)
{
    case 0:
        cout << "Square" << endl;
        break;

    case 1:
        cout << "Triangle" << endl;
        break;

    case 2:
        cout << "Circle" << endl;
        break;

    default:
        break;
}
```

7.5 Багийн ажиллагаа

1. **Гишүүдийн оролцоо**
   Э. Баттамир - **main.cpp**
   А. Баяржавхлан - **Онолын судалгаа**
   Б. Маргад-Эрдэнэ - **max.cpp**

2. **Сурсан мэдлэг**

   Багийн гишүүдийн гүйцэтгэх ажлыг тэнцүү хуваах, даалгаврыг илүү хурдан бөгөөд үр дүнтэй гүйцэтгэх боломжтойг ойлгосон. Мөн багийн гишүүд хоорондоо кодоо нэг стандартын дагуу бичих, функц болон хувьсагчийн нэршлийг ижил байдлаар ашиглах нь кодыг нэгтгэхэд хялбар болгодогийг сурсан.

   Мөн Github программ ашиглан хамтран ажиллах явцад commit болон push хийх дараалыг ойлгож, багийн гишүүдийн кодыг нэг төсөлд нэгтгэх туршлага хуримтлуулсан.

7.6 Дүгнэлт

Энэхүү лабораторийн ажлаар объект хандалтат програмчлалын үндсэн ойлголтууд болох удамшил, ерөнхийлөл, нарийвчлал, функц дахин программчлах зарчмуудыг онол болон практик жишээгээр судалж ойлголоо. Мөн эх классын параметртэй байгуулагчийг удамшсан классыг ашиглан дуудаж ажиллуулах үйлплийг практик байдлаар гүйцэтгэв. Ерөнхийллийн ойлголтыг ашиглаж Shape2D классыг хийсвэр класс болгон area, perimeter гэсэн виртуал функцуудыг тодорхойллоо. Мөн функц дахин программчлах ойлголтын хүрээнд Shape2D классаас удамшсан Circle, Triangle, Square классуудад area, perimeter функцуудад дахин программчлал хийлээ. Лабораторийн даалгаварт circle, square, triangle классуудыг ашиглан олон объект үүсгэж, тэдгээрийн талбайгаар нь эрэмбэлэх алгоритм хэрэгжүүлснээр удамшил болон полиморфизмын хэрэглээг онолын хүрээнд практик байдлаар гүйцэтгэлээ.

7.7 Ашигласан материал

1. М. Золжаргал багшийн лекцийн материал
2. Объект хандлагат технологийн C++ программчал, Ж. Пүрэв, 2008, Улаанбаатар
3. geeksforgeeks

7.8 Хавсралт

1. oop.h

```
#ifndef OOP_H
#define OOP_H

#include<string>
#include<iostream>
#include<cmath>
using namespace std;

class Shape {
   protected:
         string name;
   public:
         Shape() {};
         Shape(string name) { this->name = name; }
         void setName(const string& name);
         string getName() const;
};

class Shape2D : public Shape {
   public:
         Shape2D() : Shape() {};
         Shape2D(string name) : Shape(name) {}
         virtual double area() = 0;
         virtual double perimeter() = 0;
};

class Circle : public Shape2D {
   private:
         double crd_1;
         double crd_2;
         double radius;
   public:
         Circle() : Shape2D("Circle") {
               radius = 0;
               crd_1 = 0;
               crd_2 = 0;
         }
         Circle(double r) : Shape2D("Circle") {
               radius = r;
         }

         double area() override;
         double perimeter() override;
```

```
        void printName();
};

class Square : public Shape2D {
    private:
        double x0, y0;
        double x1, y1;
        double x2, y2;
        double x3, y3;
        double side;
    public:
        Square() : Shape2D("Square") {
            side = 0;
        }

        Square(double s) : Shape2D("Square") {
            side = s;
        }

        double area() override;
        double perimeter() override;
        void printName();
};

class Triangle : public Shape2D {
    private:
        double x0, y0;
        double x1, y1;
        double x2, y2;
    public:
        Triangle() : Shape2D("Triangle") {
            side = 0;
        }

        Triangle(double a) : Shape2D("Triangle") {
            side = a;
        }

        double area() override;
        double perimeter() override;
        void printName();
};

int max(double *areas, int n);

#endif
```

2. circle.cpp

```
#include "../oop.h"

double Circle::setter(const double n, const double cx) {

    radius = n;

    crd_1 = cx;
    crd_2 = cx;

    return radius;
}

void Circle::getter() {
    cout << "Radius: " << radius << endl;
}

double Circle::area() {
    return M_PI * radius * radius;
}

double Circle::perimeter() {
    return 2 * M_PI * radius;
}

void Circle::printName() {
    cout << "Shape name: " << name << endl;
}

void Circle::showCoordinate() {
    cout << "Center: (" << crd_1 << ", " << crd_2 << ")" << endl;
}
```

3. Max.cpp

```
#include "../oop.h"

int max(double *areas, int n) {
    int maxidx = 0;
    for (int i = 1; i < n; i++ ) {
            if (*(areas + i) > *(areas + maxidx)) maxidx = i;
    }

    return maxidx;
}
```

4. shape.cpp

```
#include "../oop.h"

void Shape::setName(const string& name) {
   this->name = name;
}

string Shape::getName() const {
   return name;
}
```

5. Square.cpp

```
#include "../oop.h"

double Square::setter(const double n, const double cx, const double
cy) {

   side = n;

   x0 = cx;
   y0 = cy;

   x1 = cx + side;
   y1 = cy;

   x2 = cx + side;
   y2 = cy - side;

   x3 = cx;
   y3 = cy - side;

   return side;
}

void Square::getter() {
   cout << "Side: " << side << endl;
}

double Square::area() {
   return side * side;
}

double Square::perimeter() {
   return 4 * side;
}
```

```
void Square::printName() {
   cout << "Shape name: " << name << endl;
}

void Square::ShowCoordinates() {

   cout << "Coordinates:" << endl;

   cout << "A(" << x0 << ", " << y0 << ")" << endl;
   cout << "B(" << x1 << ", " << y1 << ")" << endl;
   cout << "C(" << x2 << ", " << y2 << ")" << endl;
   cout << "D(" << x3 << ", " << y3 << ")" << endl;

}
```

6. triangle.cpp

```
#include "../oop.h"

double Triangle::setter(const double n, const double cx, const
double cy) {

   side = n;

   x0 = cx;
   y0 = cy;

   x1 = cx - side/2;
   y1 = cy - (sqrt(3)/2)*side;

   x2 = cx + side/2;
   y2 = cy - (sqrt(3)/2)*side;

   return side;
}

void Triangle::getter() {
   cout << "Side: " << side << endl;
}

double Triangle::area() {
   return (sqrt(3)/4) * side * side;
}

double Triangle::perimeter() {
   return 3 * side;
}
```

```
void Triangle::printName() {
    cout << "Shape name: " << name << endl;
}

void Triangle::showCoordinates() {

    cout << "Coordinates:" << endl;

    cout << "A(" << x0 << ", " << y0 << ")" << endl;
    cout << "B(" << x1 << ", " << y1 << ")" << endl;
    cout << "C(" << x2 << ", " << y2 << ")" << endl;
}
```

7. main.cpp

```
#include "oop.h"

int main () {
    int n = 3;
    Shape2D *shapes[n] = {new Square(3), new Triangle(3), new Circle(3)};
    double* k = new double[n];

    for (int i = 0; i < n; i++) {
            k[i] = shapes[i]->area();
    }

    int x = max(k, n);

    cout << "Hamgiin tom ni ";

    switch (x)
    {
            case 0:
            cout << "Square" << endl;
            break;

            case 1:
            cout << "Triangle" << endl;
            break;

            case 2:
            cout << "Circle" << endl;
            break;

            default:
            break;
    }
```

```
    for (int i = 0; i < n; i++) {
            delete shapes[i];
    }
    delete[] k;

    return 0;
}
```

8. MakeFile

```
all: go

circle.o: cpp/circle.cpp oop.h
      g++ -c cpp/circle.cpp -o circle.o

max.o: cpp/max.cpp oop.h
      g++ -c cpp/max.cpp -o max.o

shape.o: cpp/shape.cpp oop.h
      g++ -c cpp/shape.cpp -o shape.o

shape2d.o: cpp/shape2d.cpp oop.h
      g++ -c cpp/shape2d.cpp -o shape2d.o

square.o: cpp/square.cpp oop.h
      g++ -c cpp/square.cpp -o square.o

triangle.o: cpp/triangle.cpp oop.h
      g++ -c cpp/triangle.cpp -o triangle.o

main.o: main.cpp oop.h
      g++ -c main.cpp -o main.o

go: circle.o max.o shape.o shape2d.o square.o triangle.o main.o
         g++ circle.o max.o shape.o shape2d.o square.o triangle.o
               main.o -o go

clean:
rm -v *.o go
```

Даалгавар байршиж буй холбоос: Github